Advanced Retrieval Systems

Technical Deep Dive

PDF 파일 기반 RAG 고급 기법

실무 관점으로 이해하는 RAG 파이프라인의 모든 것

설계 — 추출 — 검색 — 리랭크 — 생성

Presenter

Subject

Large Language Models

Date

2026. 03. 11

Concept / RAG Fundamentals

RAG Retrieval-Augmented Generation

왜 필요한가, 무엇을 하는가

Why: LLM의 한계

LLM은 문장은 유려하지만, **학습 데이터 밖의 지식**(사내 문서, 최신 뉴스)은 모릅니다. 모르는 것도 아는 척 답하는 **환각(Hallucination)** 문제가 발생합니다.

Solution: 검색 + 생성

Retrieval
근거 문서 찾기

+

Generation
근거 기반 답변

→

정확한 답변
환각 최소화

RAG Process Flow

질문 (Query)

Search →

Retriever
관련 문서 조각 추출

Context →

LLM
근거 기반 생성

Answer →

최종 답변

품질 결정 3요소 (모델 크기보다 중요)

Extraction (추출)
문서(PDF 등)에서 텍스트를 깨끗하게 뽑아냈는가?

Chunking (청킹)
의미가 훼손되지 않도록 적절한 단위로 쪼갰는가?

Retrieval (검색)
질문에 딱 맞는 근거 조각을 찾아왔는가?

주요 실패 패턴

- **검색 실패** → 답이 틀리거나 "근거 없음" 답변
- **생성 왜곡** → 근거는 맞는데 요약/해석 실수
- **추출 불가** → 표/이미지가 깨져서 검색 원천 봉쇄

Extraction / PDF Processing

PDF 추출 방식 비교

텍스트 중심 vs 구조화 vs 시각적 이해

Basic

PyMuPDFLoader

전통적 텍스트 추출 방식

Mechanism

PDF 내부 텍스트 레이어를 읽어 **Raw Text**만 빠르게 추출

Pros & Cons

- ✓ 매우 빠르고 비용 0 (로컬)
- ✓ 대량 문서 처리에 최적
- ✗ 표/다단 편집 시 순서 섞임
- ✗ 이미지/그래프 정보 누락

추천 상황

보고서 본문 위주, 표가 적은 문서, 빠르게 베이스라인 구축할 때

Structured

PyMuPDF4LLM

LLM 친화적 마크다운 변환

Mechanism

텍스트를 **Markdown 형식**으로 변환하여 문단/헤더 구조 보존

Pros & Cons

- ✓ LLM이 이해하기 좋은 구조
- ✓ 텍스트 흐름이 자연스러움
- ✗ 복잡한 표 복원은 여전히 난해
- ✗ 문서 포맷에 따라 성능 편차

추천 상황

텍스트 위주지만 목차/구조가 중요할 때, 가독성을 높이고 싶을 때

Advanced

VLM (Vision)

이미지 기반 시각적 해석

Mechanism

페이지를 **이미지로 캡처** 후 GPT-4o 등이 시각적으로 정보 해석

Pros & Cons

- ✓ 표/그래프 수치 정확히 인식
- ✓ 레이아웃 정보(캡션 등) 보존
- ✗ 비용 높음, 속도 느림
- ✗ 작은 글씨 오인식 가능성

추천 상황

"정답이 표에 있다"가 많을 때, 재무제표/기술문서 등 그래프가 핵심일 때

Chunking Strategy / RAG Indexing

Chunking & Recursive Character Text Splitter

문맥을 보존하는 스마트한 문서 분할 전략

왜 문서를 쪼개야 하는가?

- **검색 단위 최적화:** 문서 전체는 너무 크고, 답은 특정 문단에 있음
- **LLM 제한:** Context Window 한계 및 비용 효율성

동작 원리: 재귀적 분할 (Recursive)

"의미 단위"를 최대한 유지하기 위해 큰 구분자부터 순차적으로 시도하며 자릅니다.

1. 문단 단위 (\n\n) Priority High

가장 먼저 문단 간격으로 잘라본다. 여전히 크다면?

2. 문장 단위 (\n)

줄바꿈 기준으로 나눈다. 그래도 크다면?

3. 단어/문자 (" ", "")

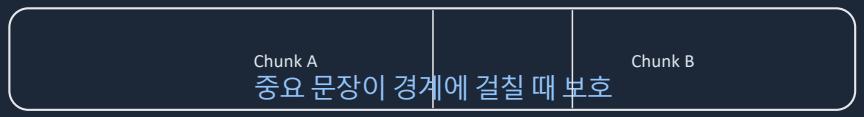
공백이나 문자 단위로 쪼개어 chunk_size를 맞춘다.

핵심 파라미터 설계

Too Small (파편화) — Too Large (정밀도↓)

Chunk Size: 보통 500~1000 토큰 내외 권장. 너무 작으면 문맥이 잘리고, 너무 크면 불필요한 정보가 섞여 검색 정확도가 떨어짐.

Overlap의 중요성



Overlap: 보통 10~20% (100~200자). 문장이 청크 사이에서 뚝 끊겨 의미를 잃는 것을 방지하는 안전장치.

실무 적용 Tip

일반 텍스트 문서 Easy

Size 1000, Overlap 200 설정이 국룰(Baseline). RecursiveSplitter로 충분함.

표/리스트가 많은 문서 Hard

단순 청킹 시 표가 깨짐. HeaderMarkdownSplitter를 쓰거나 표를 별도 추출하여 요약본만 청킹해야 함.

코드/법률 문서 Special

문법적 구조(함수, 조항 등)를 지키는 전용 Splitter (LanguageSplitter) 사용 권장.

Retrieval Strategy / Search Optimization

검색 방식 비교: FAISS vs BM25

의미 기반 검색과 키워드 매칭의 조화

Semantic

FAISS (Vector)

의미적 유사도 기반 검색

Mechanism

질문과 문서를 **임베딩 벡터**로 변환하여 의미적 거리가 가까운 것을 찾음

Pros & Cons

- ✓ 표현이 달라도 의미가 통하면 찾음
- ✓ "매출 감소" ≈ "실적 하락" 매칭
- ✗ 정확한 숫자, 고유명사, 약어에 취약
- ✗ "비슷하지만 틀린" 내용이 상위 노출

추천 상황

사용자의 질문 표현이 다양하거나, 추상적인 개념을 검색해야 할 때

Lexical

BM25 (Keyword)

단어 빈도 기반 확률적 매칭

Mechanism

TF(빈도)와 IDF(희소성) 점수를 기반으로 정확히 일치하는 단어를 찾음

Pros & Cons

- ✓ 숫자, 모델명, 약어 정확도 매우 높음
- ✓ 도메인 특화 용어("YoY", "EBITDA")
- ✗ 단어가 조금만 달라도 못 찾음 (동의어X)
- ✗ 문맥 파악 불가능

추천 상황

부품 번호, 사번, 특정 수치 등 정확한 키워드 매칭이 핵심일 때

Hybrid

Ensemble (RRF)

순위 기반 결합 전략

Mechanism

FAISS와 BM25의 결과 순위를 **재조정(RRF)**하여 최적의 결과를 도출

Pros & Cons

- ✓ 의미와 키워드 매칭의 장점 결합
- ✓ 정답 문서가 포함될 확률(Recall) 극대화
- ✗ 구현 복잡도 및 리소스 소폭 증가
- ✗ Top-k 설정 튜닝 필요

Best Practice

대부분의 실무 RAG 시스템에서 **가장 권장되는 기본 구성**

Multi-Query Retrieval

표현 불일치(Vocabulary Mismatch)를 극복하는 다중 쿼리 전략

문제: 표현 불일치

사용자의 언어

"이번 분기 상승 요인이 뭐야?"

문서의 언어 (매칭 실패)

"...당기 순이익 상방 압력으로 작용..."

사용자는 일상 용어를 쓰지만, 문서는 전문 용어나 다른 표현을 씁니다. 단일 쿼리는 이 간극을 넘지 못해 **Recall(재현율)**이 떨어집니다.

Query Expansion Process

Original Query
"상승 요인?"

LLM

문서 표현 1 (금융)
"이번 분기 상방 요인"

문서 표현 2 (전문 용어)
"이번 분기 업사이드(Upside)"

의미적 확장
"이번 분기 긍정적 실적 사유"

해결: 다중 쿼리 확장

Paraphrasing
LLM에게 질문을 다양한 표현으로 다시 쓰게 함

Perspective Expansion
동의어, 유의어, 구체적 예시 등으로 관점 확장

Union of Results
모든 쿼리의 검색 결과를 합집합(Union)하여 중복 제거

Trade-off 분석

Pros (장점) Recall ↑

서로 다른 키워드를 가진 청크들을 낚시바늘 여러 개로 긁어오듯 찾아낼 확률이 높아짐.

Cons (단점) Noise ↑ / Cost ↑

쿼리가 3개면 검색 비용도 3배. 엉뚱한 방향으로 확장되면 노이즈 문서가 유입됨.

실무 적용 꿀팁

- **Reranker 필수:** 늘어난 후보군(Noise)을 정리하려면 리랭커가 꼭 뒤에 붙어야 함.
- **도메인 제약:** 금융/법률은 용어가 중요하므로, LLM 프롬프트에 "핵심 키워드는 유지하라"고 지시해야 함.

Reranking Mechanism / Precision Optimization

리랭킹: Bi-Encoder vs Cross-Encoder

속도와 정확도 사이의 균형을 잡는 2단계 검색 전략

Bi-Encoder

Retrieval (1차 검색)

Q Doc

BERT BERT

Cosine Sim

- ✓ **개별 인코딩:** 질문과 문서를 따로 벡터화하여 미리 저장 가능 (Indexing).
- \+ **매우 빠름:** 수백만 문서 중에서 Top-K를 순식간에 검색.
- − **정밀도 낮음:** 문맥적 뉘앙스를 깊게 파악하지 못함.

Cross-Encoder

Reranking (2차 정렬)

[Query + Doc]

BERT (Full Attention)

Score (0~1)

- ✓ **동시 인코딩:** 질문과 문서를 섞어서(Interaction) 깊게 분석.
- \+ **매우 정확함:** 실제 답변이 될 확률을 직접 채점.
- − **느림:** 문서마다 모델을 돌려야 하므로 전체 검색엔 불가.

실무 RAG 파이프라인

1. Retrieval
Bi-Encoder / FAISS
Top-K 후보 **100개**

2. Rerank
Cross-Encoder
Top-N 정예 **5~10개**

3. Generation
LLM (GPT-4)
Final Answer

"Bi-Encoder로 넓게(Recall) 잡고, Cross-Encoder로 좁게(Precision) 거른다."

Top-N Trade-off

N이 너무 작으면 (3개 미만)
Recall Risk
정답 청크가 잘려나가 답변 실패.

N이 너무 크면 (20개 이상)
Cost & Noise Risk
LLM 입력 비용 증가 및 정보 혼탁(Lost in Middle).

Optimal Zone
보통 **Top-5 ~ Top-10** 사이가 품질과 비용의 최적점(Sweet Spot).

마무리: 실무 체크리스트와 다음 단계

성공적인 RAG 구축을 위한 최종 점검

RAG 구축 핵심 체크리스트

1. Extraction (추출)

문서 성격에 따라 **PyMuPDF**(텍스트 위주) vs **VLM**(표/그래프 핵심)을 명확히 선택했는가?

* 표가 중요하다면 전용 파서나 VLM 필수 고려

2. Chunking (청킹)

의미 단위가 보존되도록 **Recursive Splitter**를 사용하고, 적절한 Overlap을 설정했는가?

* 표/리스트가 청크 경계에서 깨지지 않도록 별도 처리했는지 확인

3. Retrieval (검색)

FAISS(의미)와 **BM25**(키워드)를 결합한 **Ensemble(RRF)** 전략을 적용했는가?

* Recall(재현율) 확보가 우선, 정답이 후보군에 없으면 답변 불가

4. Reranking (재정렬)

검색된 후보를 **Cross-Encoder**로 정밀 타격하여 **Top-N**을 선별했는가?

* Top-N 갯수 튜닝을 통해 비용과 품질 최적화

설계 원칙 (Design Principles)

Recall (놓치지 않기) — **Precision (정확히 집기)**

Ensemble로 넓게 잡고 Reranker로 좁히는 균형

Cost / Latency — **Performance**

실시간성 요구사항에 따른 모델 크기 타협

다음 단계 (Next Steps)

1. **v0 프로토타입 구축**
   베이스라인(PyMuPDF + Ensemble) 파이프라인 완성
2. **품질 지표 설정**
   Recall@k, Answerable@k 등 정량 지표 정의
3. **A/B 실험 & 고도화**
   Chunk Size 변경, VLM 도입 등 성능 비교 실험